

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and
Commerce Department of Information Technology (B.Sc.I.T Semester VI)
Business Analytics
Practical V

Roll No. : T052	Name: Divyanjali yadav
Class : TYIT	Date/Time of Submission : 21/08/2026

a. Simulate a probability experiment (e.g., rolling dice) using programming and calculate the probabilities of different outcomes.

Code: import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

1. Simulate 1,000 dice rolls and display the first 20 outcomes.

Code: # Set seed for reproducible results

np.random.seed(42)

Simulate 1000 dice rolls

dice_rolls = np.random.randint(1, 7, 1000)

Display first 20 outcomes

print("First 20 Dice Rolls:")

print(dice_rolls[:20])

```
First 20 Dice Rolls:
[4 5 3 5 5 2 3 3 3 5 4 3 6 5 2 4 6 6 2 4]
```

2. Calculate the experimental probability of getting each number from 1 to 6.

Code: # Calculate frequency of each outcome

frequency = pd.Series(dice_rolls).value_counts().sort_index()

Experimental probability

experimental_probability = frequency / 1000

print("Frequency of each outcome:")

print(frequency)

print("\nExperimental Probability:")

print(experimental_probability)

```
Frequency of each outcome:
```

```
1    181
2    164
3    154
4    174
5    172
6    155
Name: count, dtype: int64
```

```
Experimental Probability:
```

```
1    0.181
2    0.164
3    0.154
4    0.174
5    0.172
6    0.155
Name: count, dtype: float64
```

3. Calculate the theoretical probability of getting each number from 1 to 6.

Code: theoretical_probability = pd.Series(

[1/6] * 6,

index=[1, 2, 3, 4, 5, 6]

)

print("Theoretical Probability:")

print(theoretical_probability)

```
Theoretical Probability:
```

```
1    0.166667
2    0.166667
3    0.166667
4    0.166667
5    0.166667
6    0.166667
dtype: float64
```

4. Find the probability of getting 1, 2, 3, 4, 5, 6.

5. Find the probability of getting an even number.

6. Find the probability of getting an odd number.

7. Find the probability of getting a number greater than 4.

8. Find the probability of getting a number less than 4.

9. Find the probability of getting a number greater than or equal to 4.

Code: # Probability of each number

print("Probability of getting 1:", experimental_probability[1])

print("Probability of getting 2:", experimental_probability[2])

```

print("Probability of getting 3:", experimental_probability[3])
print("Probability of getting 4:", experimental_probability[4])
print("Probability of getting 5:", experimental_probability[5])
print("Probability of getting 6:", experimental_probability[6])
# Experimental probabilities of different events
prob_even = np.mean(dice_rolls % 2 == 0)
prob_odd = np.mean(dice_rolls % 2 != 0)
prob_greater_4 = np.mean(dice_rolls > 4)
prob_less_4 = np.mean(dice_rolls < 4)
prob_greater_equal_4 = np.mean(dice_rolls >= 4)
print("\nProbability of Even Number:", prob_even)
print("Probability of Odd Number:", prob_odd)
print("Probability Greater than 4:", prob_greater_4)
print("Probability Less than 4:", prob_less_4)
print("Probability Greater than or Equal to 4:", prob_greater_equal_4)

```

```

Probability of getting 1: 0.181
Probability of getting 2: 0.164
Probability of getting 3: 0.154
Probability of getting 4: 0.174
Probability of getting 5: 0.172
Probability of getting 6: 0.155

Probability of Even Number: 0.493
Probability of Odd Number: 0.507
Probability Greater than 4: 0.327
Probability Less than 4: 0.499
Probability Greater than or Equal to 4: 0.501

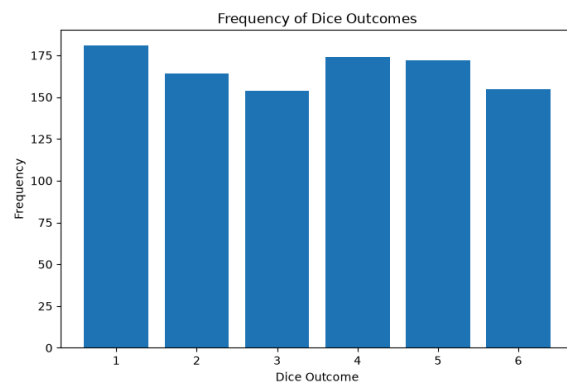
```

10. Create a bar chart showing the frequency of outcomes 1 to 6.

```

Code: plt.figure(figsize=(8, 5))
plt.bar(frequency.index, frequency.values)
plt.title("Frequency of Dice Outcomes")
plt.xlabel("Dice Outcome")
plt.ylabel("Frequency")
plt.xticks([1, 2, 3, 4, 5, 6])
plt.show()

```



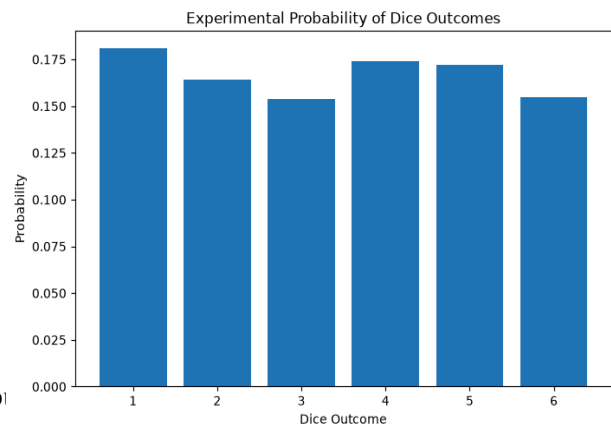
11. Create a bar chart showing the experimental probability of outcomes 1 to 6.

```

Code: plt.figure(figsize=(8, 5))
plt.bar(
    experimental_probability.index,
    experimental_probability.values
)
plt.title("Experimental Probability of Dice Outcomes")
plt.xlabel("Dice Outcome")
plt.ylabel("Probability")
plt.xticks([1, 2, 3, 4, 5, 6])

plt.show()

```

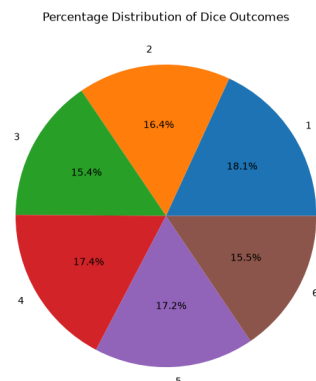


12. Create a pie chart showing the percentage distribution of outcomes 1 to 6.

```

Code: plt.figure(figsize=(7, 7))
plt.pie(
    frequency.values,
    labels=frequency.index,
    autopct='%1.1f%%'
)
plt.title("Percentage Distribution of Dice Outcomes")
plt.show()

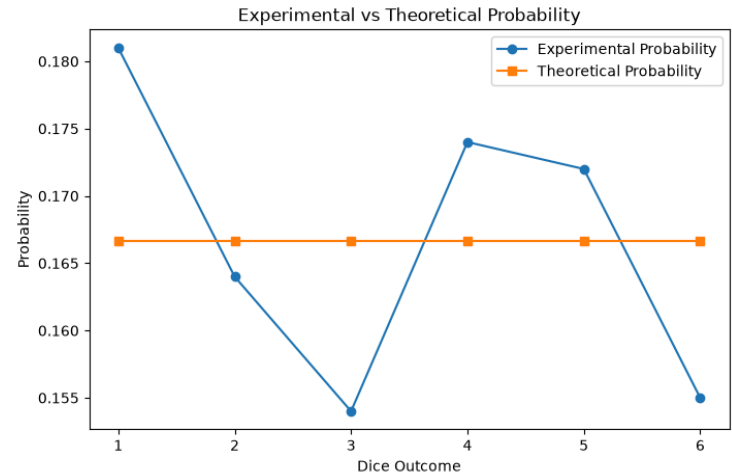
```



13. Create a line chart comparing experimental probability and theoretical probability for outcomes 1 to 6.

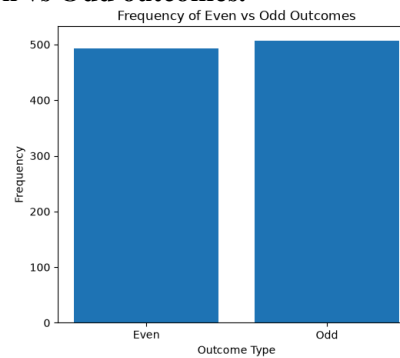
Code:plt.figure(figsize=(8, 5))

```
plt.plot(
    experimental_probability.index,
    experimental_probability.values,
    marker='o',
    label='Experimental Probability'
)
plt.plot(
    theoretical_probability.index,
    theoretical_probability.values,
    marker='s',
    label='Theoretical Probability'
)
plt.title("Experimental vs Theoretical Probability")
plt.xlabel("Dice Outcome")
plt.ylabel("Probability")
plt.xticks([1, 2, 3, 4, 5, 6])
plt.legend()
plt.show()
```



14. Create a chart showing the frequency of Even vs Odd outcomes.

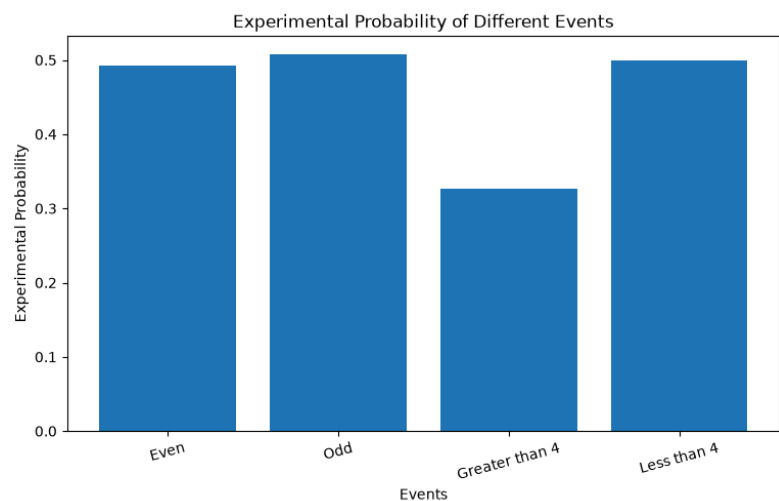
```
Code:even_count = np.sum(dice_rolls % 2 == 0)
odd_count = np.sum(dice_rolls % 2 != 0)
plt.figure(figsize=(6, 5))
plt.bar(
    ["Even", "Odd"],
    [even_count, odd_count]
)
plt.title("Frequency of Even vs Odd Outcomes")
plt.xlabel("Outcome Type")
plt.ylabel("Frequency")
plt.show()
```



15. Create a chart showing the experimental probability of:

- Even numbers
- Odd numbers
- Numbers greater than 4
- Numbers less than 4

```
Code:event_names = [
    "Even",
    "Odd",
    "Greater than 4",
    "Less than 4"
]
event_probabilities = [
    prob_even,
    prob_odd,
    prob_greater_4,
    prob_less_4
]
plt.figure(figsize=(9, 5))
plt.bar(event_names, event_probabilities)
```



```
plt.title("Experimental Probability of Different Events")
plt.xlabel("Events")
plt.ylabel("Experimental Probability")
plt.xticks(rotation=15)
plt.show()
```

b. Generate random numbers from various probability distributions (normal, uniform, exponential) and analyze their properties.

1. Generate 1,000 random numbers from a normal distribution with:

Mean = 50 and Standard deviation = 10

Code: # Generate 1000 random numbers from Normal Distribution

```
normal_data = np.random.normal(loc=50, scale=10, size=1000)
```

```
print("First 20 values:")
```

```
print(normal_data[:20])
```

```
First 20 values:
[43.49357431 45.12874616 44.07606076 41.3600923  50.48521628 41.69049884
 52.70456826 49.49761891 47.61051953 40.92436338 44.23228669 57.55391226
 55.00917188 40.22444755 50.99332305 57.51387123 33.38594719 55.43360192
 43.37376241 55.70598669]
```

2. Calculate the mean, median, mode, variance, and standard deviation of the generated data.

Code: # Calculate statistics

```
mean_value = np.mean(normal_data)
```

```
median_value = np.median(normal_data)
```

```
# Calculate mode
```

```
mode_value = pd.Series(normal_data).mode()
```

```
variance_value = np.var(normal_data)
```

```
std_value = np.std(normal_data)
```

```
print("Mean:", mean_value)
```

```
print("Median:", median_value)
```

```
print("Mode:", mode_value.values)
```

```
print("Variance:", variance_value)
```

```
print("Standard Deviation:", std_value)
```

```
Mean: 50.67560929616276
Median: 50.58115779455787
Mode: [21.03744622 21.51457379 23.03113357 23.49030192 24.08957708 24.46078865
 25.00594285 25.283555 25.76120673 27.88864601 28.01194043 28.46609886
 28.47109407 29.18070592 29.26609768 29.58265132 29.61875465 30.01799315
 30.06264357 30.24533434 30.33643411 31.10459269 31.29208079 31.59125769
 31.63794627 31.95138789 31.98019560 32.21279951 32.39191188 32.39237241
 32.60286221 32.89811087 32.97416396 33.07043186 33.0753537 33.30594719
 33.31415926 33.38479938 33.39030066 33.45143328 33.54601253 33.72457562
 33.86438602 34.07006266 34.16097177 34.23607843 34.29499396 34.2977528
 34.43378826 34.43418101 34.44104354 34.74343685 34.74474829 34.76813095
 34.84255885 34.96857047 35.14439627 35.15102032 35.21088431 35.21413754
 35.3562512 35.510861 35.59949117 35.69224898 35.78189333 35.81634267
 35.86285081 35.91538784 35.92488305 35.93138903 35.9704728 36.10427533
 36.20600772 36.20808857 36.3019702 36.33121302 36.40143858 36.48315304
 36.51814578 36.5287371 36.55549489 36.64055641 36.65074505 36.68767046
 36.76910207 36.88163768 37.03168052 37.04921228 37.05318524 37.100391
 37.17007776 37.19570648 37.19695601 37.20423033 37.23251424 37.48460576
 37.48886424 37.55345297 37.56136765 37.72392185 37.74234337 37.75701764
 37.76059733 37.77872191 37.889838 37.90385257 37.90358981 38.04116938
 38.12401581 38.29880674 38.41635311 38.53054607 38.50095007 38.70293145
 38.70402229 38.74511953 38.77277978 38.80330105 38.89424155 38.93474091
 38.96410683 38.96633304 38.96791634 38.93724913 39.07687235 39.18936672
 39.1894346 39.29915234 39.34886337 39.46594437 39.53089017 39.55190806
 39.60094073 39.73484701 39.77207435 39.78767183 39.85243269 39.87895625
 ...
 74.4575198 74.5530014 74.92999517 75.26932426 75.58199286 75.60084538
 75.73359083 75.79709138 76.01683114 76.32382065]
Variance: 98.318994801817
Standard Deviation: 9.915593517375397
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

3. Calculate the following statistics for the generated

Normal distribution:

- Mean
- Median
- Variance
- Standard deviation
- Minimum
- Maximum

Code: print("Statistics of Normal Distribution")

```
print("-----")
```

```
print("Mean:", np.mean(normal_data))
```

```
print("Median:", np.median(normal_data))
```

```
print("Variance:", np.var(normal_data))
```

```
print("Standard Deviation:", np.std(normal_data))
```

```
print("Minimum:", np.min(normal_data))
```

```
print("Maximum:", np.max(normal_data))
```

```
Statistics of Normal Distribution
-----
Mean: 50.67560929616276
Median: 50.58115779455787
Variance: 98.318994801817
Standard Deviation: 9.915593517375397
Minimum: 21.037446218063106
Maximum: 76.32382064837391
```

4. Create a histogram of the Normal distribution and analyze its shape.

Code: plt.figure(figsize=(8, 5))

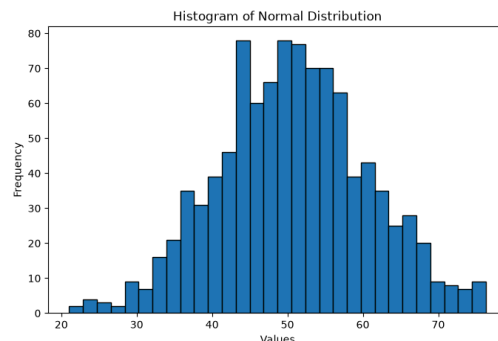
```
plt.hist(normal_data, bins=30, edgecolor="black")
```

```
plt.title("Histogram of Normal Distribution")
```

```
plt.xlabel("Values")
```

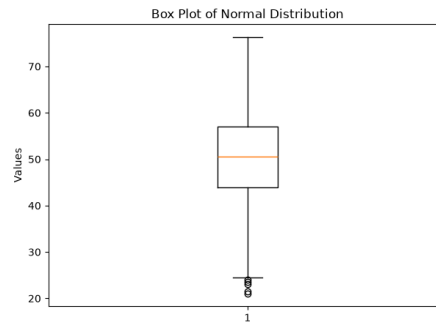
```
plt.ylabel("Frequency")
```

```
plt.show()
```



5. Create a box plot of the Normal distribution.

```
Code: plt.figure(figsize=(7, 5))
plt.boxplot(normal_data)
plt.title("Box Plot of Normal Distribution")
plt.ylabel("Values")
plt.show()
```

**6. Calculate the percentage of values that fall between: $40 \leq X \leq 60$**

```
Code: percentage_40_60 = np.mean(
    (normal_data >= 40) & (normal_data <= 60)
) * 100
```

```
print("Percentage of values between 40 and 60:",
      percentage_40_60, "%")
```

```
Percentage of values between 40 and 60: 68.2 %
```

7. Calculate the percentage of values that fall between: $30 \leq X \leq 70$

```
Code: percentage_30_70 = np.mean(
    (normal_data >= 30) & (normal_data <= 70)
) * 100
```

```
print("Percentage of values between 30 and 70:",
      percentage_30_70, "%")
```

```
Percentage of values between 30 and 70: 95.1 %
```

8. Generate 1,000 random numbers from a uniform distribution between 0 and 100.

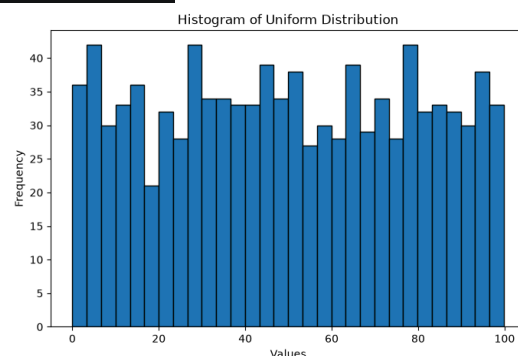
```
Code: uniform_data = np.random.uniform(
    low=0,
    high=100,
    size=1000
)
```

```
First 20 values:
[ 5.85509292 33.42383187 78.48969771 70.76809347 78.86149647 51.72690564
 44.0198998  14.74525267 32.81927536 43.40193662  8.86004297 22.06119527
 59.82252941 73.56631142 99.83475114 93.31133313 64.25651995 42.12480533
 63.61773647 78.56516181]
```

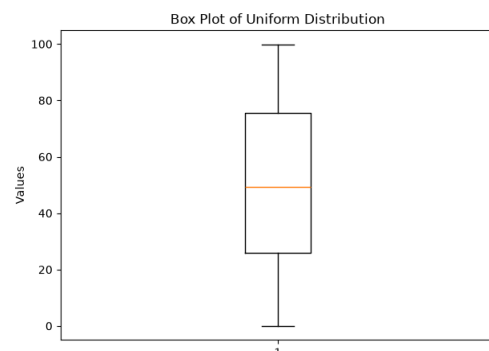
```
print("First 20 values:")
print(uniform_data[:20])
```

9. Create a histogram of the Uniform distribution.

```
Code: plt.figure(figsize=(8, 5))
plt.hist(uniform_data, bins=30, edgecolor="black")
plt.title("Histogram of Uniform Distribution")
plt.xlabel("Values")
plt.ylabel("Frequency")
plt.show()
```

**10. Create a box plot of the Uniform distribution.**

```
Code: plt.figure(figsize=(7, 5))
plt.boxplot(uniform_data)
plt.title("Box Plot of Uniform Distribution")
plt.ylabel("Values")
plt.show()
```

**11. Generate 1,000 random numbers from an Exponential distribution with scale = 2.**

```
Code: exponential_data = np.random.exponential(
    scale=2,
    size=1000
)
```

```
print("First 20 values:")
print(exponential_data[:20])
```

```
First 20 values:
[0.61665721 1.84036599 0.70354514 2.50649532 2.84596053 0.21631989
 1.44249449 1.42217655 0.92178895 5.40388367 3.51478936 2.38942748
 2.50581241 1.23873849 5.07647615 2.37223583 2.61113315 3.95652854
 0.64060775 3.29086507]
```

12. Create a histogram of the Exponential distribution.

```
Code: plt.figure(figsize=(8, 5))
```

```
plt.hist(exponential_data, bins=30, edgecolor="black")
```

```
plt.title("Histogram of Exponential Distribution")
plt.xlabel("Values")
plt.ylabel("Frequency")
```

```
plt.show()
```

13. Create a box plot and identify possible extreme values

```
Code: plt.figure(figsize=(7, 5))
plt.boxplot(exponential_data)
plt.title("Box Plot of Exponential Distribution")
plt.ylabel("Values")
```

```
plt.show()
```

14. Calculate the percentage of values greater than 2.

```
Code: Q1 = np.percentile(exponential_data, 25)
Q3 = np.percentile(exponential_data, 75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
outliers = exponential_data[
    (exponential_data < lower_bound) |
    (exponential_data > upper_bound)
]
```

```
print("Possible Extreme Values:")
```

```
print(outliers)
```

```
percentage_greater_2 = np.mean(
    exponential_data > 2
) * 100
```

```
print("Percentage of values greater than 2:",
      percentage_greater_2, "%")
```

15. Calculate the percentage of values greater than 5.

```
percentage_greater_5 = np.mean(
    exponential_data > 5
) * 100
```

```
print("Percentage of values greater than 5:",
      percentage_greater_5, "%")
```

C) Coin toss task (Include Prints)

1. Simulate 1,000 coin tosses and calculate the experimental probability of:

• Getting Heads

• Getting Tails

```
Code: # Set seed for reproducible results
```

```
np.random.seed(42)
```

```
# Simulate 1000 coin tosses
```

```
coin_tosses = np.random.choice(["Heads", "Tails"], size=1000)
```

```
print("First 20 Coin Tosses:")
```

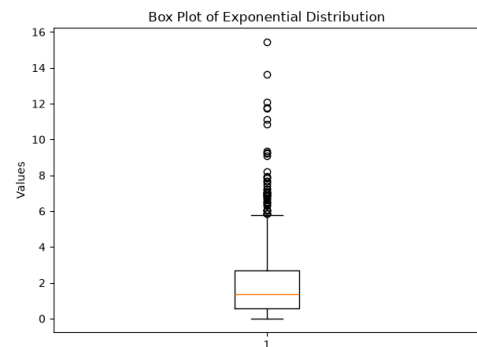
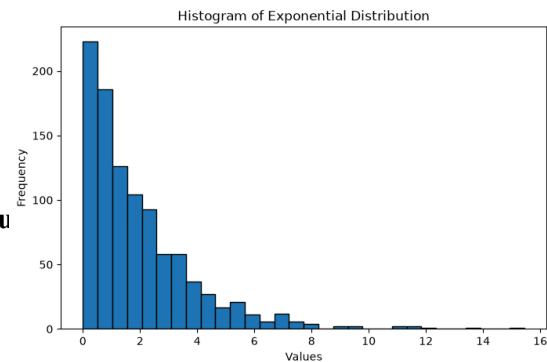
```
print(coin_tosses[:20])
```

```
# Calculate frequency
```

```
coin_frequency = pd.Series(coin_tosses).value_counts()
```

```
# Experimental probability
```

```
coin_probability = coin_frequency / 1000
```



```
Possible Extreme Values:
[ 7.68177524  5.87835106  7.03033032  6.82463447  7.86126427  7.05873517
  6.91790714  6.04921931  6.70569288  5.86490392  6.32031852  7.54512695
  6.05199991  6.10735919  6.43090352  6.88815624 11.74370692 11.12335447
13.61150244 12.08829437  9.35100442 10.86649238  7.93908262  6.48429642
11.78352797  7.25238172  7.40539971  7.22953367  6.86918562  9.10347325
  6.05010902  7.00822347 15.44705904  6.51379539  5.88102897  6.61324697
  9.23222893  6.8888183  7.88169154  8.21695268  6.99477566  7.09135348
  6.3457987  7.66594192  6.7479745  5.8657495  9.27793048]
```

Percentage of values greater than 2: 36.8 %

Percentage of values greater than 5: 7.5 %

```
First 20 Coin Tosses:
```

```
['Heads' 'Tails' 'Heads' 'Heads' 'Heads' 'Tails' 'Heads' 'Heads' 'Heads'
 'Tails' 'Heads' 'Heads' 'Heads' 'Heads' 'Tails' 'Heads' 'Tails' 'Tails'
 'Tails' 'Heads']
```

```
Frequency:
```

```
Tails    510
```

```
Heads    490
```

```
Name: count, dtype: int64
```

```
Experimental Probability:
```

```
Tails    0.51
```

```
Heads    0.49
```

```
Name: count, dtype: float64
```

```
print("Frequency:")
print(coin_frequency)
print("\nExperimental Probability:")
print(coin_probability)
```

2. Calculate the theoretical probability of Heads and Tails.

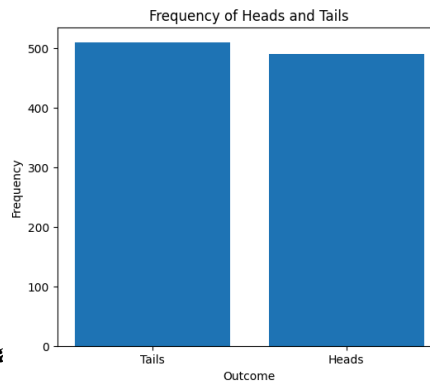
```
theoretical_coin_probability = {
    "Heads": 0.5,
    "Tails": 0.5
}
```

```
Theoretical Probability:
{'Heads': 0.5, 'Tails': 0.5}
```

```
print("Theoretical Probability:")
print(theoretical_coin_probability)
```

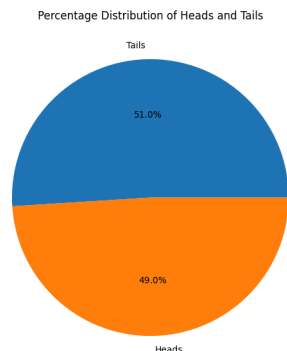
3. Create a bar chart showing the frequency of Heads and Tails.

```
plt.figure(figsize=(6, 5))
plt.bar(
    coin_frequency.index,
    coin_frequency.values
)
plt.title("Frequency of Heads and Tails")
plt.xlabel("Outcome")
plt.ylabel("Frequency")
plt.show()
```



4. Create a pie chart showing the percent:

```
plt.figure(figsize=(7, 7))
plt.pie(
    coin_frequency.values,
    labels=coin_frequency.index,
    autopct="%1.1f%%"
)
plt.title("Percentage Distribution of Heads and Tails")
plt.show()
```



5. If a coin is tossed 10 times and Heads occurs 8 times, calculate the experimental probability of Heads.

Code:- import math

n = 10

k = 8

p = 0.5

```
probability_8_heads = (
    math.comb(n, k)
    * (p ** k)
    * ((1 - p) ** (n - k))
)
```

```
print("Probability of getting exactly 8 Heads in 10 tosses:")
print(probability_8_heads)
```

```
Probability of getting exactly 8 Heads in 10 tosses:
0.0439453125
```

6. Simulate 10,000 coin tosses and create a complete analysis containing:

- Total tosses
- Number of Heads
- Number of Tails

- Experimental probability of Heads
- Experimental probability of Tails
- Theoretical probabilities

```
Code:- np.random.seed(42)
coin_tosses_10000 = np.random.choice(
    ["Heads", "Tails"],
    size=10000
)
# Frequency
frequency_10000 = pd.Series(
    coin_tosses_10000
).value_counts()
# Experimental probability
probability_10000 = frequency_10000 / 10000
print("Frequency for 10,000 Coin Tosses:")
print(frequency_10000)
print("\nExperimental Probability:")
print(probability_10000)
```

```
Frequency for 10,000 Coin Tosses:
Heads    5013
Tails    4987
Name: count, dtype: int64

Experimental Probability:
Heads    0.5013
Tails    0.4987
Name: count, dtype: float64
```

- Difference between experimental and theoretical probabilities
- Bar chart
- Pie chart

```
Code:- comparison = pd.Series({
    "Head": 0.5,
    "Tail": 0.5
})
comparison.plot(
    kind="bar",
    figsize=(8, 5)
)
plt.title("Comparison of Coin Toss Probabilities")
plt.xlabel("Outcome")
plt.ylabel("Probability")
plt.show()
```

